VARDHAMAN
COLLEGE OF ENGINEERING

A Case-Study Report on

MINI DATABASE ENGINE USING C++ AND DATA STRUCTURES

DATA STRUCTURES : A9503

Submitted by

25881A0549: T. SAI PURNANAND

25881A0562: G. YASHWANTH

25881A0563: D. YOGESH

Course Facilitator

Dr. V. Lokeshwari Vinya

Associate Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

June 2026

CONTENTS

Case Study 1: Computer Functional Units

Case Study 1: Computer Functional Units

1. Introduction

The rapid growth of data-driven applications in modern computing has made database systems an essential component of software engineering. From small-scale applications to large enterprise systems, databases are used to store, manage, and retrieve data efficiently. However, most users interact only with high-level database systems without understanding the internal mechanisms that enable efficient data storage and querying. This case study focuses on developing a Mini Database Engine using fundamental data structures in C++, providing insights into how core database operations are implemented at a low level.

The system simulates basic database functionalities such as insertion, deletion, searching, and updating of records. Instead of relying on existing database management systems, the project builds these features from scratch using data structures like linked lists and hash maps. This approach helps in understanding how data organization and indexing significantly impact system performance. By implementing a simplified database engine, the study bridges the gap between theoretical concepts and real-world applications, making it highly relevant for computer science and engineering students.

1.1 Importance and Relevance to Engineering and Society

In today's digital world, efficient data management is critical for almost every domain, including healthcare, banking, education, and e-commerce. Database systems form the backbone of these applications by ensuring that data is stored securely and retrieved quickly. Engineers must understand how these systems work internally to design scalable and efficient solutions. This case study provides valuable insights into the role of data structures in building such systems.

From a societal perspective, database systems enable seamless access to information, improve decision-making, and enhance user experiences. For example, online platforms rely on efficient data retrieval to provide instant results to users. By studying a Mini Database Engine, students gain a deeper understanding of how real-world systems manage large volumes of data. This knowledge is essential for developing innovative solutions that can handle increasing data demands while maintaining performance and reliability. Thus, the project is both academically significant and practically relevant.

1.2 Objectives of the Case Study

The objectives of this case study are:

- To design and implement a Mini Database Engine using C++ and fundamental data structures.

- To demonstrate efficient implementation of core database operations such as insertion, deletion, searching, and updating without using external database systems.
- To explore the use of linked lists for dynamic data storage.
- To utilize hash maps for fast and efficient data retrieval.
- To highlight the importance of indexing in improving system performance.
- To develop a modular system design where components like storage, indexing, parsing, and execution can be implemented independently.
- To apply theoretical knowledge of data structures to solve practical problems.
- To gain a deeper understanding of system design principles and real-world database functionality.

2. Case Study Background

The concept of database systems has evolved significantly over the years, from simple file-based storage systems to complex relational and non-relational databases. Modern database management systems provide advanced features such as concurrency control, transaction management, and distributed storage. However, the fundamental principles of data storage and retrieval remain rooted in basic data structures and algorithms.

This case study is based on the idea of simplifying these complex systems into a manageable and educational model. By developing a Mini Database Engine, the project focuses on core functionalities that are essential for understanding how databases work internally. The system is designed to store records dynamically, perform efficient searches, and handle user queries in a structured manner. The use of data structures such as linked lists and hash maps ensures that the system is both efficient and scalable.

The project also emphasizes teamwork and modular development, allowing each team member to focus on a specific component of the system. This approach not only improves productivity but also reflects real-world software development practices. By studying this case, students gain practical experience in designing and implementing data-driven systems.

2.1 Description of the Case Scenario

The case scenario involves the development of a Mini Database Engine that simulates the basic functionalities of a real database system. The system is designed to handle a set of records, where each record contains fields such as ID, name, and other attributes. Users can interact with the system through simple commands to perform operations like inserting new records, searching for existing records, updating data, and deleting records.

The system operates in a command-driven environment, where user inputs are parsed and executed by the engine. For example, a user can enter a command to insert a new record, and the system will store the data using a linked list while updating the hash map for indexing. Similarly, search operations are performed using the hash map to ensure fast retrieval of data.

This scenario represents a simplified version of real-world database systems, where efficient data handling is crucial for performance. By implementing this system, the project demonstrates how different components work together to achieve efficient data management. The case study provides a practical understanding of how database engines process queries and manage data internally.

2.2 Key Facts, Figures, and Stakeholders Involved

Key Facts:

- The Mini Database Engine is developed using fundamental data structures in C++
- The system simulates core database operations such as insertion, deletion, searching, and updating
- Each record consists of structured attributes such as a unique identifier, name, and other fields
- Linked lists are used for dynamic memory allocation and flexible data storage
- Hash maps are used to achieve fast data retrieval with near constant time complexity
- Indexing techniques significantly improve system performance by reducing search time from linear to constant complexity
- The system is capable of handling multiple records efficiently without major performance degradation

Stakeholders Involved:

- Software developers who design and implement database systems
- System architects who focus on optimizing performance and scalability
- End users who depend on fast and reliable data retrieval
- Students and researchers who use the system as a learning model for understanding database internals

2.3 Background Information

Database systems are an essential component of modern computing, enabling efficient storage, retrieval, and management of data. Traditionally, data was stored in flat files, which lacked efficiency and scalability when handling large datasets. With the evolution of database management systems, advanced techniques such as indexing, query optimization, and structured storage were introduced to improve performance.

At the core of these systems are fundamental data structures that determine how data is organized and accessed. Linked lists provide flexibility in memory allocation, allowing dynamic insertion and deletion of records. Hash maps enable fast data retrieval by mapping keys to specific memory locations, significantly reducing search time. These concepts form the foundation of modern database systems.

In this case study, these fundamental principles are applied to develop a Mini Database Engine that mimics real-world database behavior. By understanding how data structures contribute to system efficiency, students gain valuable insights into the design and implementation of large-scale data systems. This background forms the basis for analyzing the problem and developing an effective solution.

3. Problem Identification and Analysis

The increasing demand for efficient data management systems has highlighted the limitations of traditional storage methods. In many applications, data needs to be accessed quickly and updated frequently, which requires optimized storage and retrieval mechanisms. Without proper data structures, operations such as searching, inserting, and deleting records can become inefficient, leading to performance bottlenecks.

The primary problem addressed in this case study is the need for an efficient and scalable system that can handle basic database operations while maintaining optimal performance. Simple data storage methods, such as arrays or sequential files, are not suitable for dynamic datasets where frequent modifications are required. Additionally, linear search operations can significantly increase response time as the size of the dataset grows.

This case study focuses on overcoming these challenges by using appropriate data structures to optimize performance. By integrating linked lists for dynamic storage and hash maps for fast lookup, the system aims to provide an efficient solution for managing structured data. The analysis emphasizes the importance of selecting the right data structures to achieve scalability and efficiency in real-world applications.

3.1 Main Concept: Mini Database Engine

The main concept of this case study revolves around the design and implementation of a Mini Database Engine using fundamental data structures. A database engine is responsible for storing, retrieving, and managing data efficiently, and it forms the core component of any database management system. In this project, the engine is simplified to focus on essential operations such as insertion, deletion, searching, and updating of records.

The system is designed to simulate how real database engines work internally. Each record is stored dynamically using a linked list, allowing flexibility in handling varying amounts of data. To improve search efficiency, a hash map is used as an indexing mechanism, enabling quick access to records based on unique identifiers. This combination of data structures ensures that the system can handle operations efficiently even as the dataset grows.

The Mini Database Engine serves as an educational model that demonstrates the practical application of data structures in database systems. It highlights how core concepts such as indexing and dynamic storage contribute to overall system performance and scalability.

3.2 Engineering Principles and Theories Applied

The implementation of the Mini Database Engine is based on several fundamental engineering principles and concepts from computer science. These principles ensure efficient data storage, fast retrieval, and scalable system design. By utilizing appropriate data structures and modular architecture, the system achieves flexibility, maintainability, and improved performance.

The following engineering principles are applied:

- Efficient Data Organization: Data structures such as linked lists are used to enable dynamic memory allocation, allowing the system to handle varying amounts of data without predefined limits
- Hashing and Indexing: Hash maps are used to map unique identifiers to memory locations, significantly reducing search time and improving retrieval efficiency
- Modular Design Principle: The system is divided into components such as storage, indexing, parsing, and execution, enabling independent development and easier maintenance
- Scalability Principle: The system is designed to handle increasing data efficiently without major performance degradation

- Time Complexity Optimization: The use of appropriate data structures reduces operation time complexity, especially for search and retrieval operations
- Abstraction and Encapsulation: The system hides internal implementation details and organizes functionality into logical modules for better structure and usability

3.3 Analysis of the Identified Problem

The analysis of the problem highlights the challenges associated with inefficient data management systems. In traditional approaches, data is often stored sequentially, requiring linear search operations to retrieve specific records. This leads to increased time complexity, especially when dealing with large datasets. Such inefficiencies can result in delays and reduced system performance.

In the context of the Mini Database Engine, the use of linked lists and hash maps addresses these challenges effectively. Linked lists allow dynamic insertion and deletion of records without the need for shifting elements, which is a limitation in array-based storage. Hash maps provide direct access to records, reducing search time significantly. This combination ensures that the system can handle operations efficiently even as the number of records increases.

The analysis demonstrates that the choice of data structures plays a crucial role in system performance. By selecting appropriate structures, the system achieves a balance between flexibility and efficiency. This case study reinforces the importance of data structure selection in solving real-world problems and optimizing system performance.

4. Application of Program Outcomes

This case study contributes to the following Program Outcomes (POs):

- PO1: Engineering Knowledge: Application of fundamental data structures such as linked lists and hash maps to design an efficient Mini Database Engine
- PO2: Problem Analysis: Identification of inefficiencies in traditional data storage methods and development of optimized solutions using appropriate data structures
- PO3: Design/Development of Solutions: Design and implementation of a structured and scalable data management system
- PO4: Investigation of Complex Problems: Testing and validation of operations such as insertion, deletion, and searching to ensure system reliability
- PO5: Engineering Tool Usage: Utilization of C++ programming language and standard template libraries for implementation
- PO6: The Engineer and Society: Development of efficient data management solutions applicable to real-world scenarios
- PO7: Ethics: Responsible handling and management of data within the system

- PO8: Individual and Team Work: Collaborative development of the project by dividing tasks among team members
- PO9: Communication: Effective documentation and presentation of the case study
- PO10: Project Management: Planning and execution of the project in a structured manner within given timelines
- PO11: Life-Long Learning: Encouragement of continuous learning through exploration of advanced database concepts

Program Specific Outcomes (PSOs):

- PSO1: Ability to analyze and understand the internal working of computing systems such as database engines
- PSO2: Ability to design and develop efficient software solutions using appropriate data structures and algorithms
- PSO3: Ability to apply programming and problem-solving skills to real-world data management applications

5. Sustainable Development Goals (SDGs) Related to the Case Study

5.1 SDG 9: Industry, Innovation, and Infrastructure

The Mini Database Engine developed in this case study supports technological innovation by demonstrating efficient data storage and retrieval mechanisms using fundamental data structures. Efficient database systems play a crucial role in modern industries, where large volumes of data need to be processed quickly and accurately. By implementing optimized data handling techniques such as hashing and dynamic memory allocation, the system contributes to the development of scalable and high-performance software solutions.

In real-world applications, database systems are a key component of digital infrastructure, enabling industries such as banking, healthcare, e-commerce, and education to operate efficiently. The ability to manage data effectively enhances productivity, supports decision-making, and drives innovation. This case study highlights how fundamental engineering concepts can be applied to build systems that contribute to industrial growth and technological advancement.

6. Proposed Solution and Recommendations

6.1 Proposed Solution

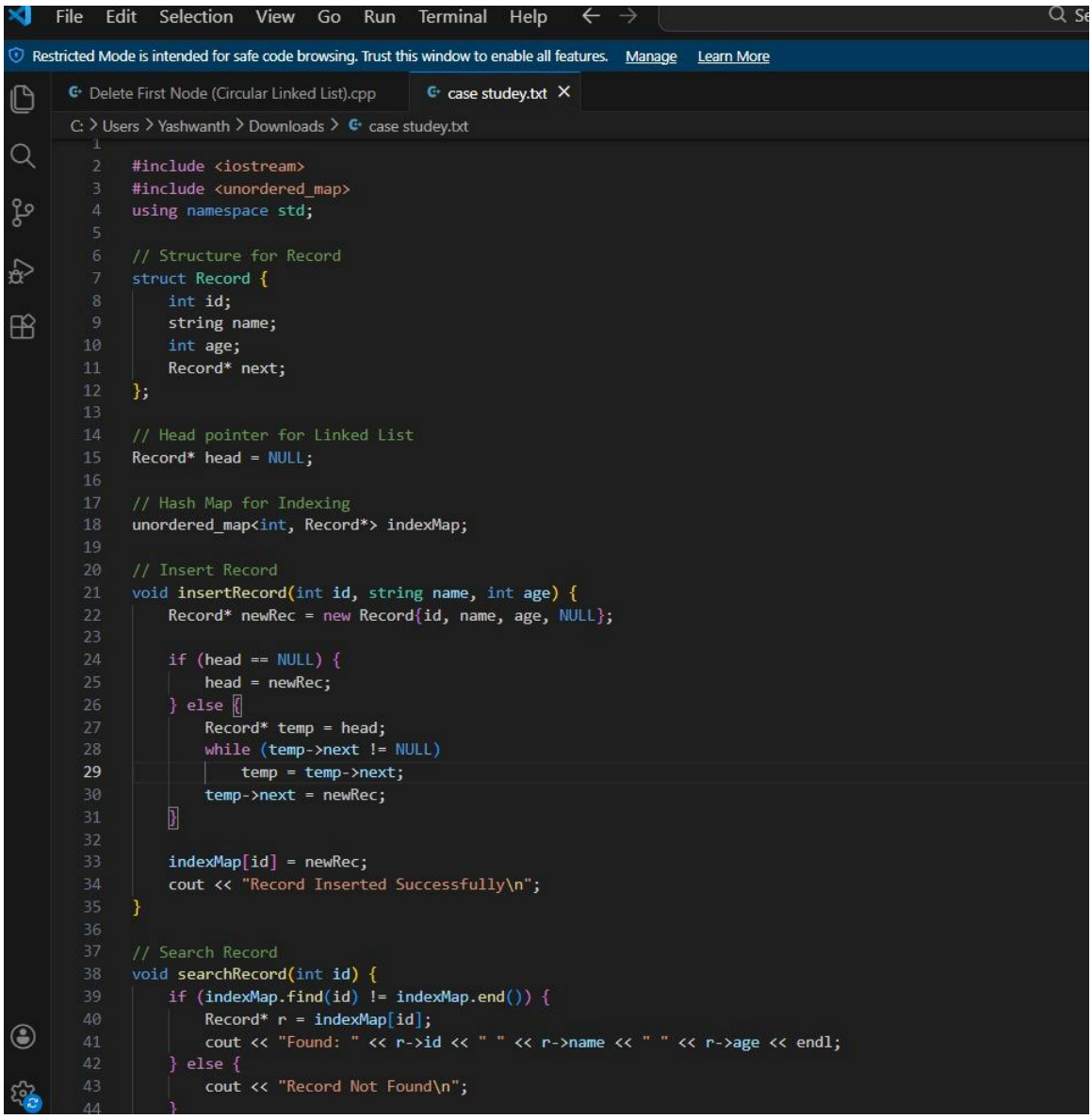
- Development of a Mini Database Engine using fundamental data structures for efficient data management.
- Implementation of core database operations such as insertion, deletion, searching, and updating of records.

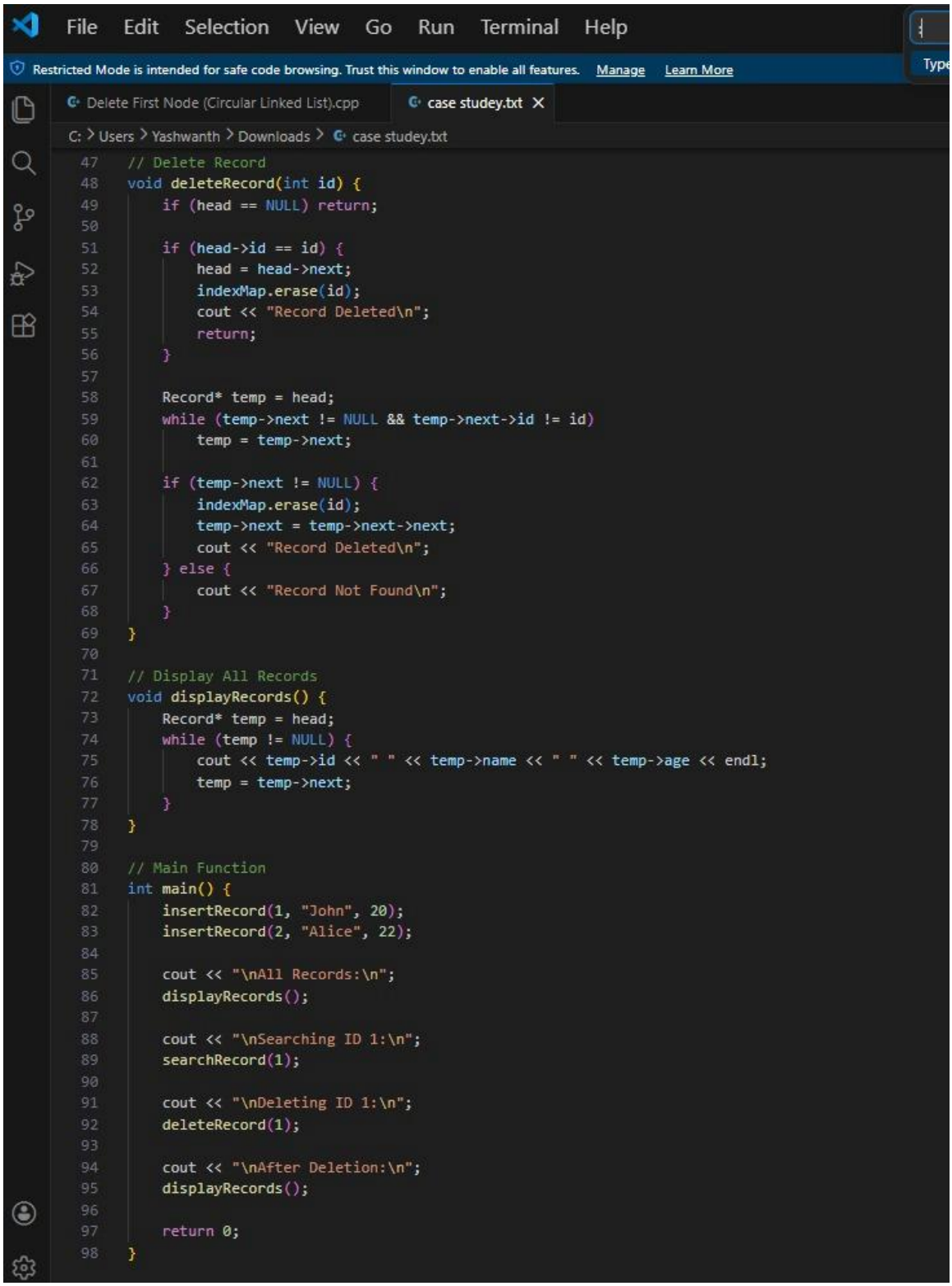
- Use of linked lists for dynamic data storage, allowing flexible memory allocation without predefined limits.
- Ensures efficient memory utilization and adaptability to varying data sizes.
- Implementation of hash map as an indexing mechanism for fast data retrieval.
- Enables quick access to records using unique identifiers, significantly reducing search time.
- Integration of linked lists and hash maps to achieve both efficiency and scalability.
- Adoption of a modular design approach with independent components such as storage, indexing, query parsing, and execution.
- Improves system maintainability and supports future enhancements.
- Provides a structured and efficient solution to the identified data management challenges.

6.2 Recommendations

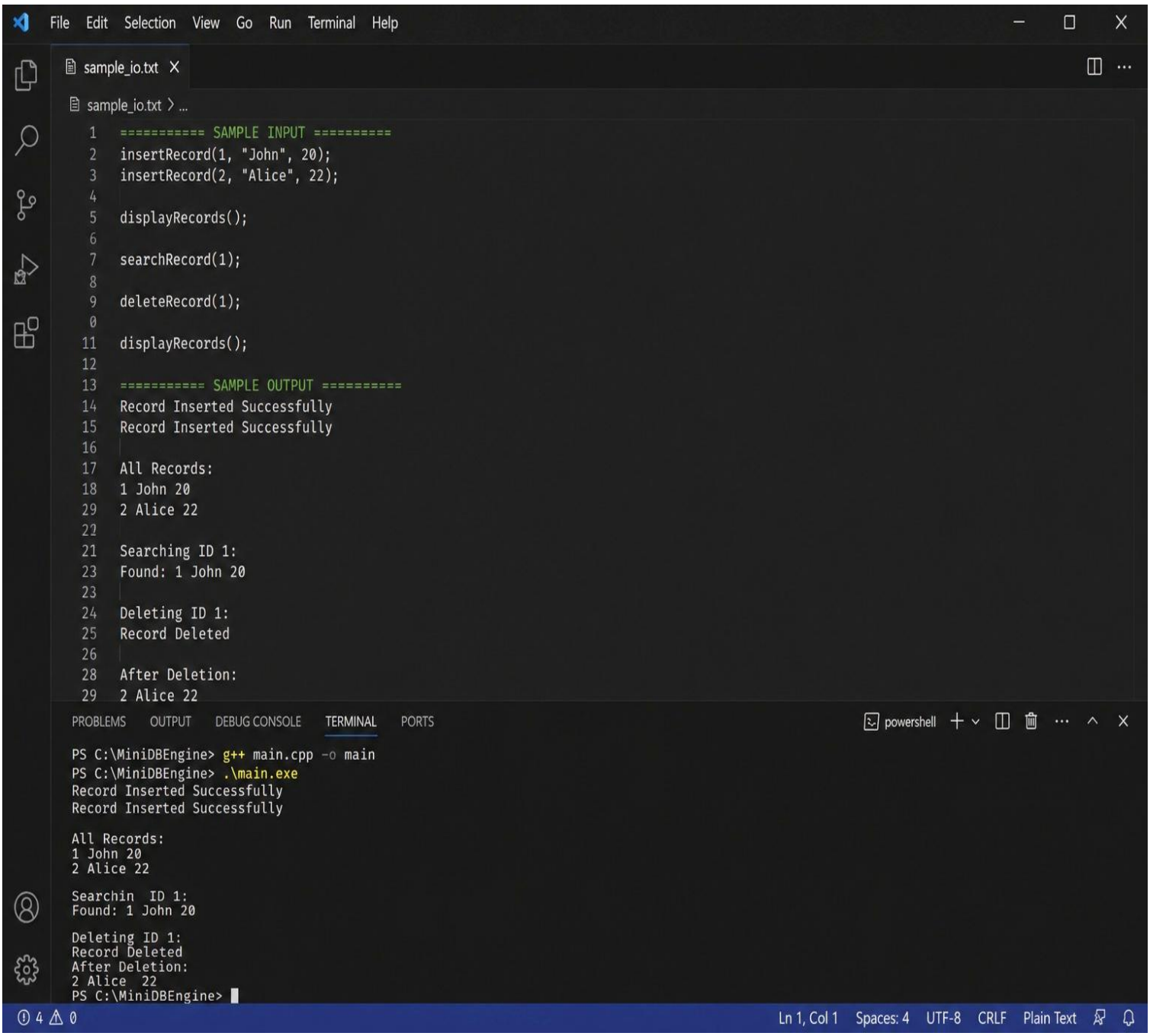
- Incorporate file handling mechanisms to enable persistent data storage beyond program execution.
- Extend the system to support multiple tables for improved functionality.
- Implement complex query handling similar to SQL for better database simulation.
- Use advanced data structures such as trees for improved indexing and performance with large datasets.
- Develop a graphical user interface to enhance usability and user interaction.
- Improve accessibility for non-technical users through better interface design.
- Integrate with modern technologies such as cloud storage for scalability.
- Explore distributed system integration for handling large-scale data efficiently.
- Enhance system features to make it more suitable for real-world applications.

6.3 Implementation Code




File Edit Selection View Go Run Terminal Help
Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More
Delete First Node (Circular Linked List).cpp
case studey.txt
C: > Users > Yashwanth > Downloads > case studey.txt
// Delete Record
void deleteRecord(int id) {
if (head == NULL) return;
if (head->id == id) {
head = head->next;
indexMap.erase(id);
cout << "Record Deleted\n";
return;
}
Record* temp = head;
while (temp->next != NULL && temp->next->id != id)
temp = temp->next;
if (temp->next != NULL) {
indexMap.erase(id);
temp->next = temp->next->next;
cout << "Record Deleted\n";
} else {
cout << "Record Not Found\n";
}
}
// Display All Records
void displayRecords() {
Record* temp = head;
while (temp != NULL) {
cout << temp->id << " " << temp->name << " " << temp->age << endl;
temp = temp->next;
}
}
// Main Function
int main() {
insertRecord(1, "John", 20);
insertRecord(2, "Alice", 22);
cout << "\nAll Records:\n";
displayRecords();
cout << "\nSearching ID 1:\n";
searchRecord(1);
cout << "\nDeleting ID 1:\n";
deleteRecord(1);
cout << "\nAfter Deletion:\n";
displayRecords();
return 0;
}

- **Sample Input and Output Format**



7. Conclusion and Lessons Learned

7.1 Conclusion

The Mini Database Engine case study successfully demonstrates the practical application of data structures in designing an efficient data management system. By integrating linked lists and hash maps, the system effectively handles dynamic data storage and fast retrieval operations. The implementation highlights the importance of selecting appropriate data structures to achieve optimal performance and scalability.

The case study achieved its objectives by providing a clear understanding of how database systems operate internally. It bridges the gap between theoretical concepts and practical implementation, making it a valuable learning experience for students. The modular design of the system ensures flexibility and allows for future enhancements, making it a strong foundation for advanced database development.

Overall, the project emphasizes the significance of data structures in solving real-world problems and demonstrates how engineering principles can be applied to develop efficient and reliable systems.

7.2 Lessons Learned

The case study provided key insights into the design and implementation of data-driven systems, highlighting how data structures can be applied to solve real-world problems efficiently. It emphasized structured design, teamwork, and the development of scalable and reliable solutions, strengthening both technical and practical knowledge.

- Selection of appropriate data structures for efficiency and flexibility
- Importance of modular design for development and maintenance
- Need for testing and validation for system reliability
- Understanding of core concepts and real-world applications
- Encouragement of continuous learning and improvement

Date: _____

Name of the Course Lead: _____
(Course Lead)